



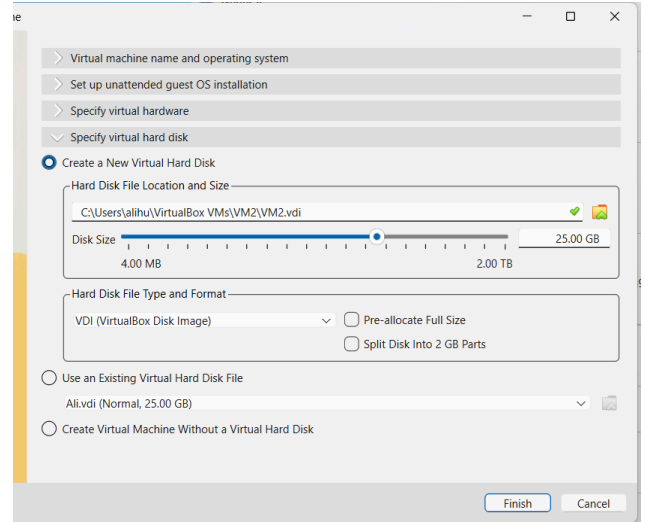
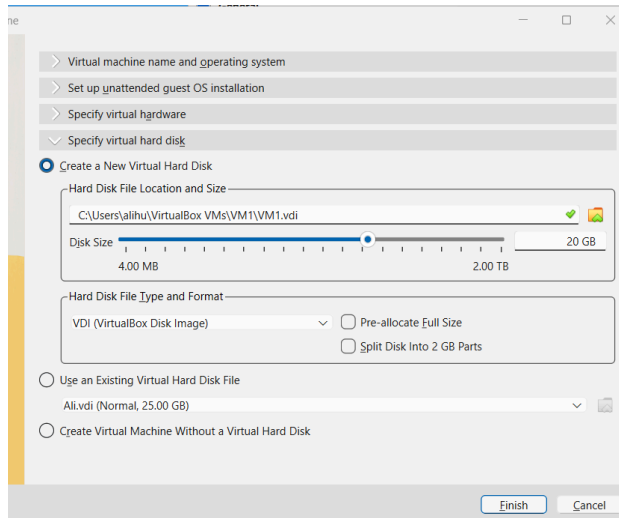
Computer Network Project

- **Student Name & ID:**

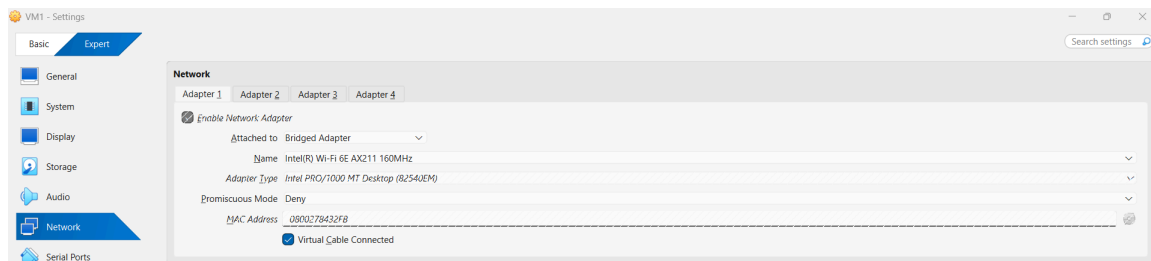
- A – Ali Hakkani: 100867456
- B – Lok Yung Chan, Carson: 100926449
- C – Harshil Joshi: 100875240
- D – Nicholas Schinkel: 100876395
- E – Rehman Leghari 100834052

Part 1

VM setup:



We configured the disk size, RAM, and other system settings for both virtual machines. This configuration was applied consistently to VM1 and VM2, and all subsequent descriptions in this report are based on this setup.



By default, the virtual machines are configured with NAT, which prevents them from communicating with each other. Therefore, the network adapter was switched to a bridged adapter, allowing both VMs to communicate directly with one another.

VM ip ping output

Server

```
harshiljoshi@harshiljoshi-VirtualBox:~$ ip a
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:41:5b:b8 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.10/24 brd 192.168.1.255 scope global enp0s3
        valid_lft forever preferred_lft forever
```

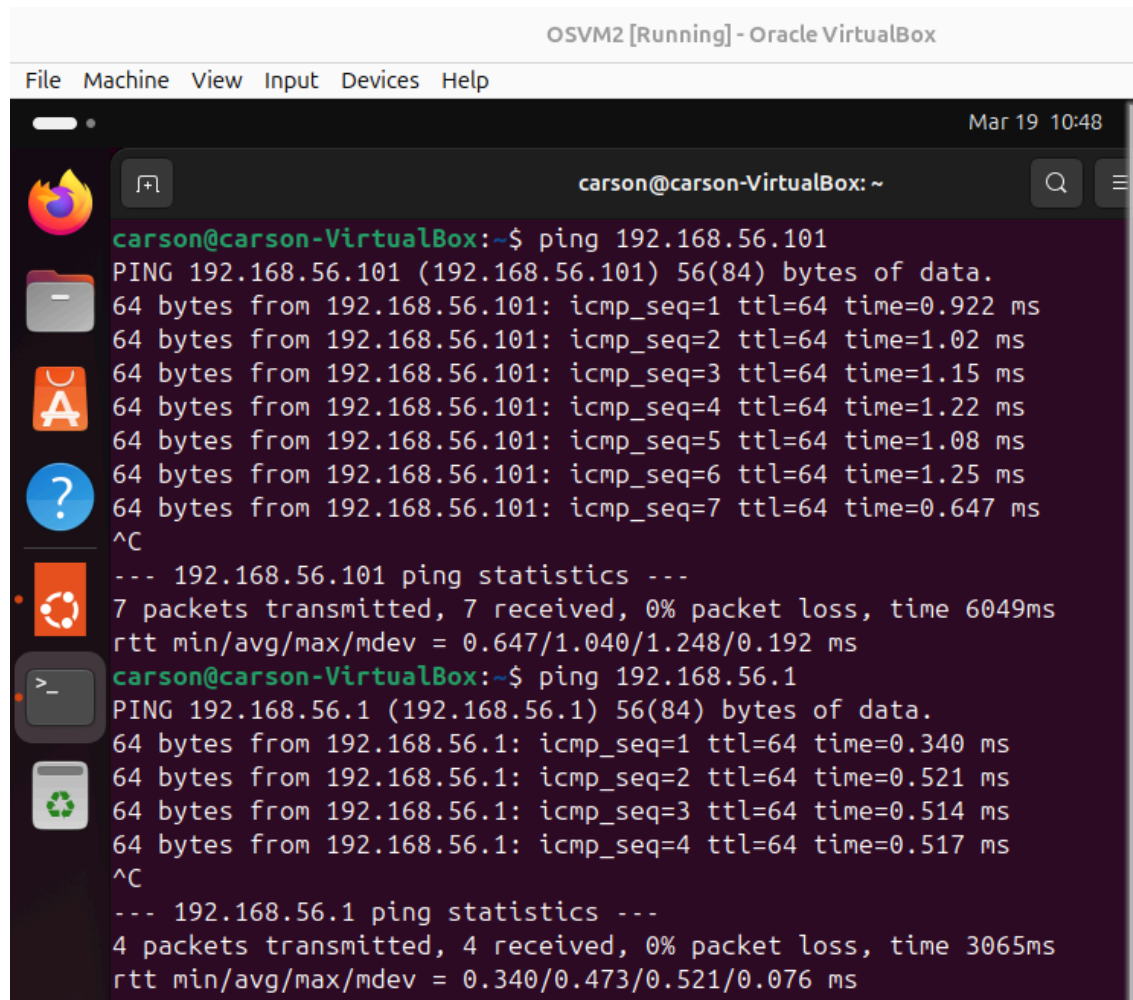
```
harshiljoshi@harshiljoshi-VirtualBox:~$ ip route
default via 192.168.1.1 dev enp0s3 proto static
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15
192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.10
harshiljoshi@harshiljoshi-VirtualBox:~$
```

Client

```
harshilclient@harshilclient-VirtualBox:~$ ip a
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:59:a7:7c brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.20/24 brd 192.168.1.255 scope global enp0s3
        valid_lft forever preferred_lft forever
```

```
harshilclient@harshilclient-VirtualBox:~$ ip route
default via 192.168.1.1 dev enp0s3 proto static
192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.20
harshilclient@harshilclient-VirtualBox:~$
```

Ping (VM1, VM2, Host):



The screenshot shows a terminal window titled "OSVM2 [Running] - Oracle VirtualBox" with a menu bar (File, Machine, View, Input, Devices, Help) and a status bar (Mar 19 10:48). The terminal prompt is "carson@carson-VirtualBox: ~". The user has executed two ping commands. The first command is "ping 192.168.56.101", which shows 7 successful pings with times ranging from 0.647 ms to 1.25 ms. The second command is "ping 192.168.56.1", which shows 4 successful pings with times ranging from 0.340 ms to 0.521 ms. Both commands show 0% packet loss.

```
carson@carson-VirtualBox:~$ ping 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.922 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=1.15 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=1.22 ms
64 bytes from 192.168.56.101: icmp_seq=5 ttl=64 time=1.08 ms
64 bytes from 192.168.56.101: icmp_seq=6 ttl=64 time=1.25 ms
64 bytes from 192.168.56.101: icmp_seq=7 ttl=64 time=0.647 ms
^C
--- 192.168.56.101 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6049ms
rtt min/avg/max/mdev = 0.647/1.040/1.248/0.192 ms
carson@carson-VirtualBox:~$ ping 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=64 time=0.340 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=64 time=0.521 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=64 time=0.514 ms
64 bytes from 192.168.56.1: icmp_seq=4 ttl=64 time=0.517 ms
^C
--- 192.168.56.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3065ms
rtt min/avg/max/mdev = 0.340/0.473/0.521/0.076 ms
```

OSVM1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Mar 19 10:48

carson@carson-VirtualBox

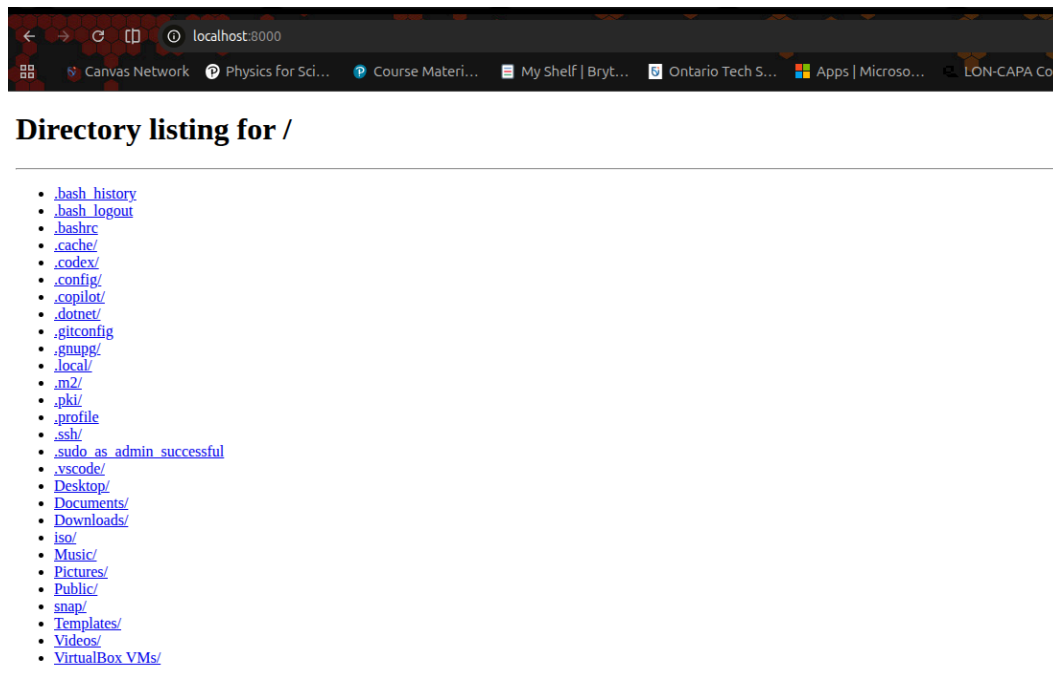
```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   link/ether 08:00:27:59:8f:16 brd ff:ff:ff:ff:ff:ff
   inet6 fe80::a00:27ff:fe59:8f16/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
   link/ether 08:00:27:41:9d:f7 brd ff:ff:ff:ff:ff:ff
   inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic
       valid_lft 563sec preferred_lft 563sec
   inet6 fe80::74d7:8e9c:8370:3d41/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
carson@carson-VirtualBox:~$ ping 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.639 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=1.33 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=1.33 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.901 ms
64 bytes from 192.168.56.102: icmp_seq=5 ttl=64 time=1.34 ms
^C
--- 192.168.56.102 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4017ms
rtt min/avg/max/mdev = 0.639/1.107/1.341/0.287 ms
carson@carson-VirtualBox:~$ ping 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
```

carson@carson-Aspire-A515-57: ~

```
valid_lft forever preferred_lft forever
carson@carson-Aspire-A515-57:~$ ping 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.558 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.691 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.621 ms
^C
--- 192.168.56.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2077ms
rtt min/avg/max/mdev = 0.558/0.623/0.691/0.054 ms
carson@carson-Aspire-A515-57:~$ ping 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.771 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.692 ms
```

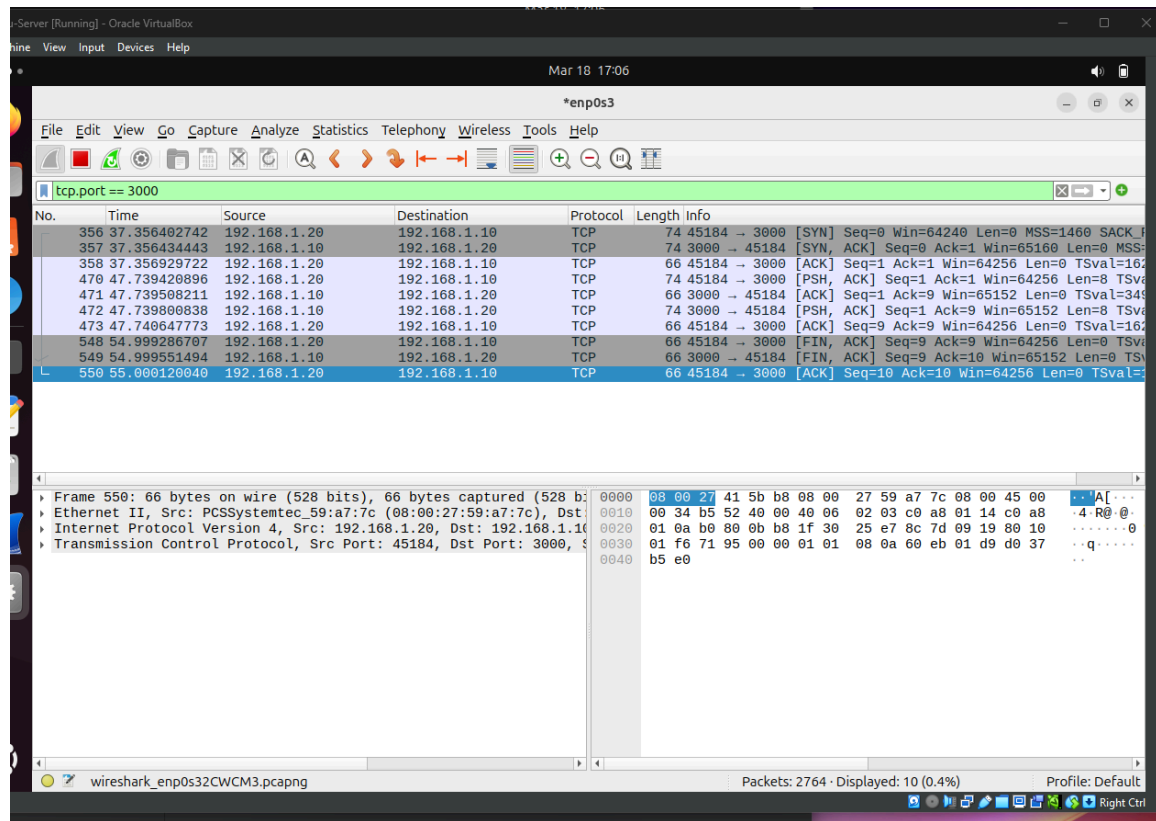
Both VMs are able to ping each other and the host after enabling bridge adapter and host-only adapter in settings.

Web Server Screenshot:



Server created in host using `$Python3 -m http.server 8000`.

Wireshark Captured Data:



Wireshark packet capture showing the TCP 3-way handshake (SYN, SYN-ACK, ACK) during connection establishment, and the FIN/ACK packets during connection termination .

Part 2

TCP Connection Packets and Termination Packets

From the Wireshark screenshot in part 1, the connection packets are those with SYN and ACK tags in the Info column, while termination packets are those with FIN tag.

Concurrent Server Verification

```
harshiljoshi@harshiljoshi-VirtualBox: ~  
harshiljoshi@harshiljoshi-V... x harshiljoshi@harshiljoshi-V... x harshiljoshi@harshiljoshi-V..  
harshiljoshi@harshiljoshi-VirtualBox:~$ ps -a  
  PID TTY          TIME CMD  
  1720 tty2      00:00:00 gnome-session-b  
  4546 pts/0      00:00:00 sudo  
  4548 pts/2      00:00:15 wireshark  
  4601 pts/2      00:00:00 dumpcap  
  4631 pts/1      00:00:00 echo_server  
  4747 pts/1      00:00:00 echo_server  
  4762 pts/3      00:00:00 ps  
harshiljoshi@harshiljoshi-VirtualBox:~$
```

Executing `ps -a` demonstrates the concurrent server successfully running multiple `echo_server` child processes simultaneously to handle multiple clients.

“Hello” Application

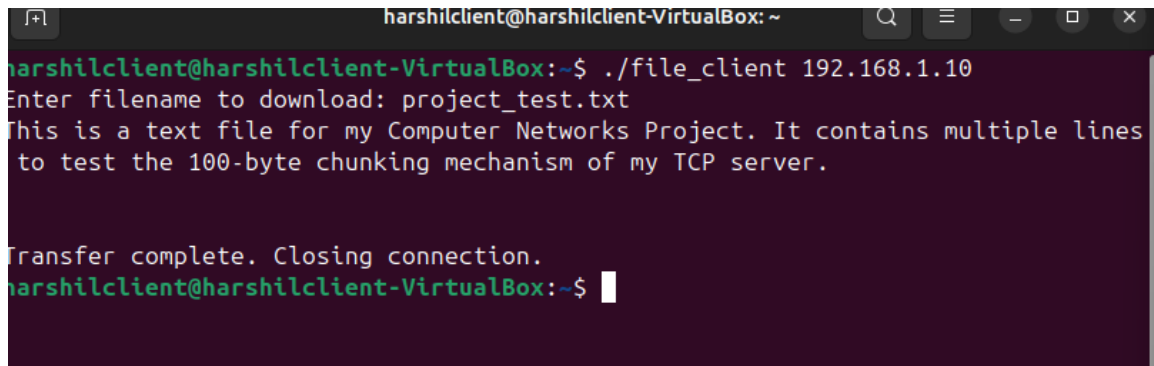
```
harshiljoshi@harshiljoshi-VirtualBox: ~  
harshiljoshi@harshiljoshi-VirtualBox: ~ x harshiljoshi@harshiljoshi-VirtualBox: ~ x  
harshiljoshi@harshiljoshi-VirtualBox:~$ gcc -o hello_server hello_server.c  
harshiljoshi@harshiljoshi-VirtualBox:~$ ./hello_server  
Server is waiting on port 3000...  
Client connected! Sending Hello...  
  
harshilclient@harshilclient-VirtualBox: ~  
harshilclient@harshilclient-VirtualBox:~$ gcc -o hello_client hello_client.c  
harshilclient@harshilclient-VirtualBox:~$ ./hello_client  
Usage: ./hello_client 192.168.1.10  
harshilclient@harshilclient-VirtualBox:~$ ./hello_client 192.168.1.10  
Message from server: Hello  
Connection closed by server. Exiting.  
harshilclient@harshilclient-VirtualBox:~$
```

Successful execution of the custom TCP "Hello" application. The server accepts the connection and transmits the "Hello" string before gracefully closing the connection

Part 3:

Successful File Download (Client & Server)

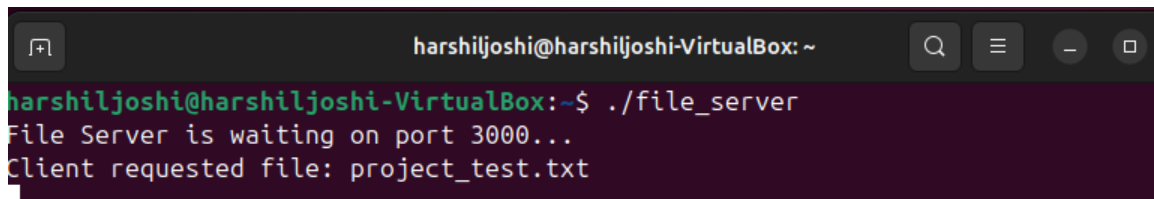
project_test.txt



```
harshilclient@harshilclient-VirtualBox: ~  
harshilclient@harshilclient-VirtualBox:~$ ./file_client 192.168.1.10  
Enter filename to download: project_test.txt  
This is a text file for my Computer Networks Project. It contains multiple lines  
to test the 100-byte chunking mechanism of my TCP server.  
  
Transfer complete. Closing connection.  
harshilclient@harshilclient-VirtualBox:~$
```

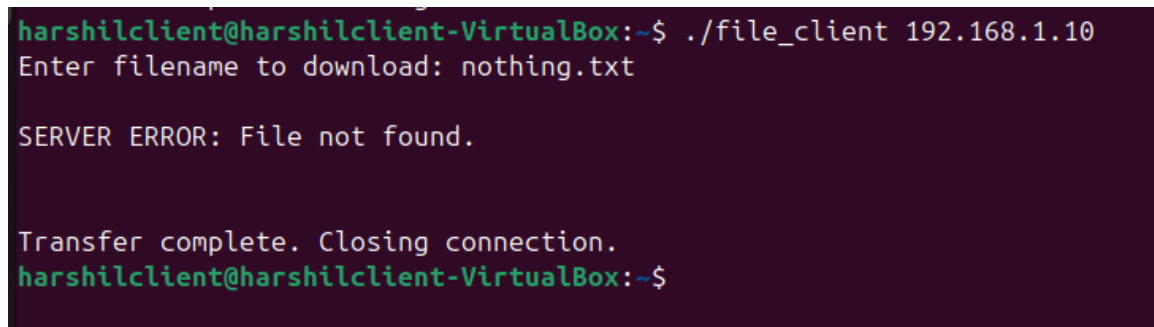
The TCP client successfully requests and downloads project_test.txt. The server transmits the file data in 100-byte chunks, and the client reconstructs it locally.

Request Processing (Server Side)



```
harshiljoshi@harshiljoshi-VirtualBox: ~  
harshiljoshi@harshiljoshi-VirtualBox:~$ ./file_server  
File Server is waiting on port 3000...  
Client requested file: project_test.txt
```

Error Handling (Client side)



```
harshilclient@harshilclient-VirtualBox:~$ ./file_client 192.168.1.10  
Enter filename to download: nothing.txt  
  
SERVER ERROR: File not found.  
  
Transfer complete. Closing connection.  
harshilclient@harshilclient-VirtualBox:~$
```

Demonstration of the TCP application's error handling. The client requests a non-existent file, and the server successfully transmits the "SERVER ERROR: File "not found" message.

Part 4

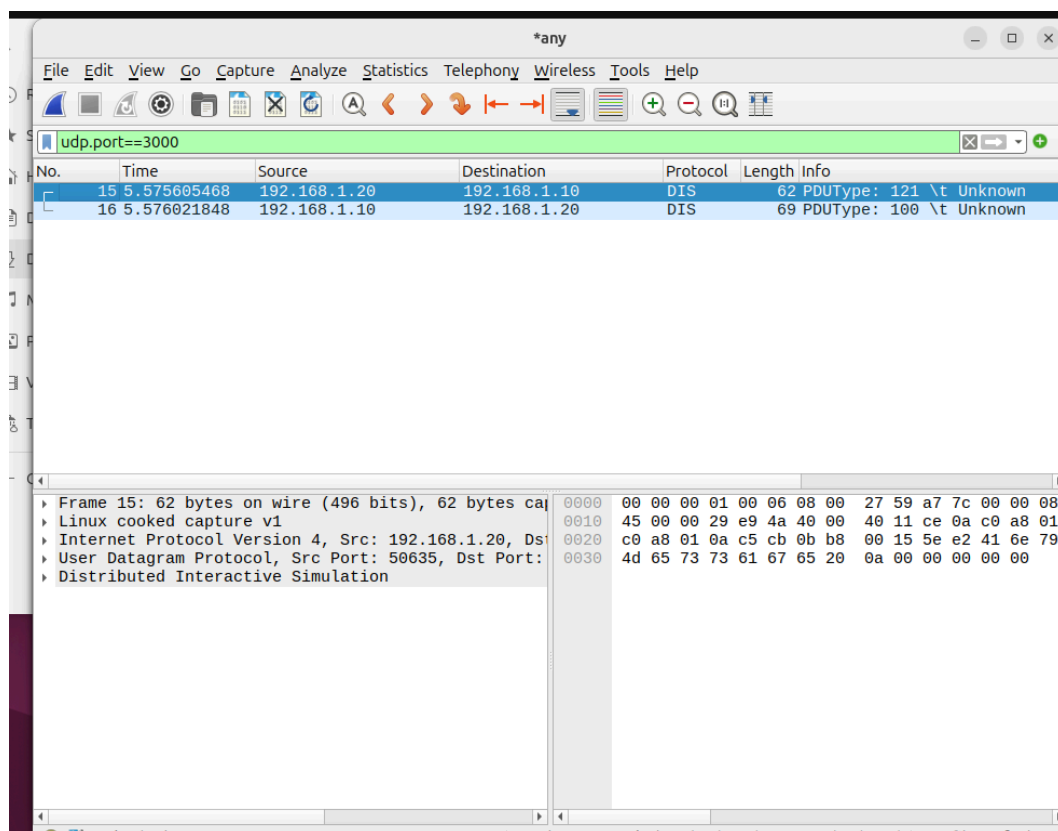
Time Service Execution

```
harshiljoshi@harshiljoshi-VirtualBox: ~  
harshiljoshi@harshiljoshi-VirtualBox:~$ gcc -o time_server time_server.c  
harshiljoshi@harshiljoshi-VirtualBox:~$ ./time_server 3000
```

```
harshilclient@harshilclient-VirtualBox:~$ gcc -o time_client time_client.c  
harshilclient@harshilclient-VirtualBox:~$ ./time_client 192.168.1.10 3000  
Wed Mar 18 23:18:15 2026  
harshilclient@harshilclient-VirtualBox:~$
```

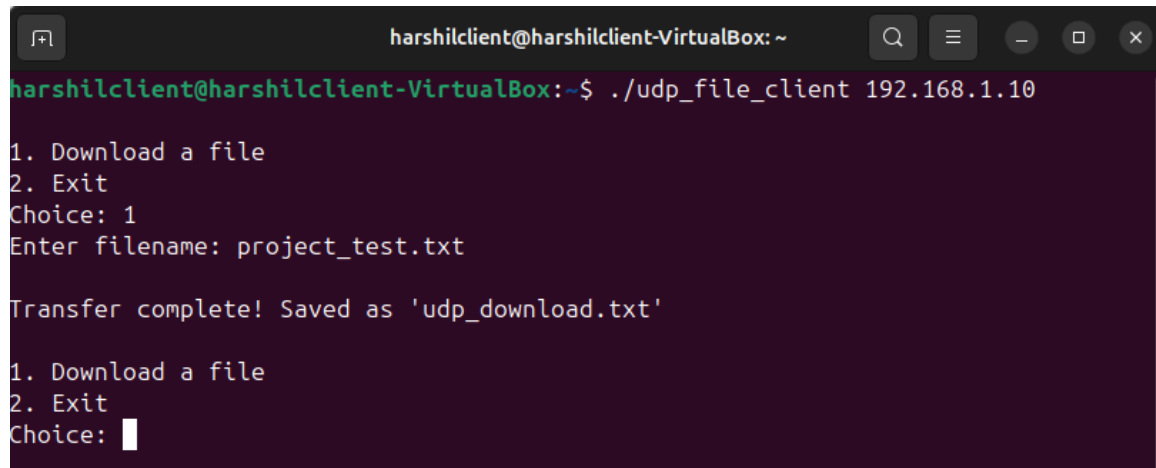
Execution of the provided iterative UDP Time Service, with the client successfully receiving the current system time from the server .

Time Service (Wireshark Capture)



Wireshark capture of the UDP Time Service. Exactly two UDP packets are exchanged: the initial client trigger message, and the server's time response. (Note: Wireshark defaults to labeling port 3000 as DIS protocol, but the frame details confirm it is UDP) .

Successful Download (Client Side)

A terminal window titled 'harshilclient@harshilclient-VirtualBox: ~' with search, menu, and window control icons. The prompt is 'harshilclient@harshilclient-VirtualBox:~\$'. The command './udp_file_client 192.168.1.10' has been executed. The program displays a menu: '1. Download a file', '2. Exit', and 'Choice: 1'. It then prompts 'Enter filename: project_test.txt'. After the command is entered, it shows 'Transfer complete! Saved as 'udp_download.txt''. The menu is shown again, with 'Choice:' followed by a cursor.

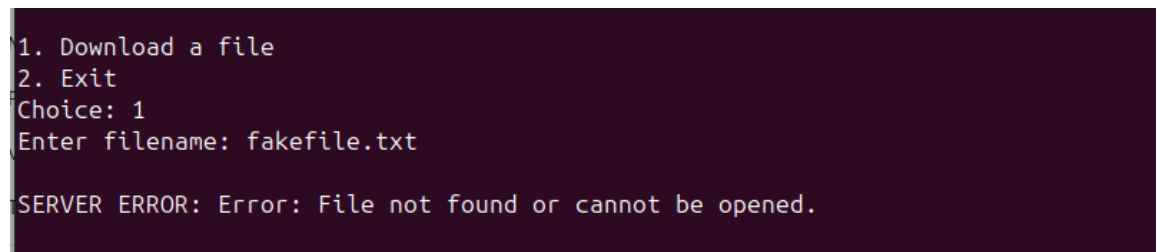
```
harshilclient@harshilclient-VirtualBox:~$ ./udp_file_client 192.168.1.10
1. Download a file
2. Exit
Choice: 1
Enter filename: project_test.txt

Transfer complete! Saved as 'udp_download.txt'

1. Download a file
2. Exit
Choice: █
```

Execution of the UDP File Download client using the custom struct pdu. The interactive menu is used to request project_test.txt (via a Filename PDU), and the client successfully receives and saves the file chunks (Data and Final PDUs) .

Error Handling (Client Side)

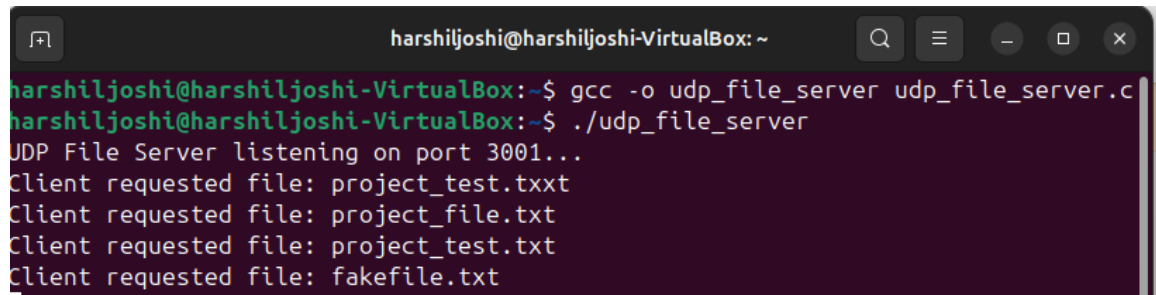
A terminal window showing the same menu as the previous screenshot. The user selects 'Choice: 1' and enters 'fakefile.txt' at the 'Enter filename:' prompt. The program then outputs 'SERVER ERROR: Error: File not found or cannot be opened.'.

```
1. Download a file
2. Exit
Choice: 1
Enter filename: fakefile.txt

SERVER ERROR: Error: File not found or cannot be opened.
```

Demonstration of error handling in the UDP client. When a non-existent file (fakefile.txt) is requested, the client correctly parses the incoming Error PDU from the server and displays the appropriate "SERVER ERROR" message

Request Processing (Server Side)

A screenshot of a terminal window titled 'harshiljoshi@harshiljoshi-VirtualBox: ~'. The terminal shows the following commands and output:

```
harshiljoshi@harshiljoshi-VirtualBox:~$ gcc -o udp_file_server udp_file_server.c
harshiljoshi@harshiljoshi-VirtualBox:~$ ./udp_file_server
UDP File Server listening on port 3001...
Client requested file: project_test.txxt
Client requested file: project_file.txt
Client requested file: project_test.txt
Client requested file: fakefile.txt
```

The UDP File Server listens on port 3001 and processes incoming client requests. The server successfully extracts the requested filenames from the incoming PDUs to either transmit the file data or return an error message .

Question

By examining the captured data, you probably find that the first message was sent by the client. This message contained some random message that was ignored by the server. What is the function of this message?

The first message from the client serves as a handshake message between it and the server, allowing the server to become aware of it and therefore establishes a communication session.

Discussion:

Which transport protocol is more appropriate for the Time service, UDP or TCP?

TCP is a connection-oriented protocol, it requires the client and server to do a “handshake” and create a session before performing any data exchange. It is also a more reliable delivery of data by enabling error checking, ordered transmission and retransmission of lost packets.

The UDP, on the other hand, is a connectionless transport protocol. It does not establish a session before data exchange, nor does a handshake with the client to guarantee delivery, which is also why it is much faster than TCP.

For a Time service, TCP is the more appropriate protocol. This is because TCP provides a more accurate and reliable transport of time information, which is essential for synchronization and scheduling of the system. However, in cases where transmission is cheap, UDP protocol may be preferred for overall lower latency.

Should the Time server be designed as a concurrent server or non-concurrent server?

In this project, the time server is responsible for tracking time, handling delay and simulating timeout and scheduling, so that the system can detect communication issues and react accordingly.

In this sense, a timer server should be designed as a concurrent server if the system has a large user base, because it has to manage communications with multiple clients at the same time. Being a concurrent server prevents the system from being blocked while it is handling a communication issue of a client. This is especially important in handling time-sensitive events. However, for a smaller scale system, where communication happens less often, a non-concurrent time server would be sufficient.

Code

1. Part 2: "Hello" Application

```
hello_server.c

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define SERVER_TCP_PORT 3000

int main() {
    int sd, new_sd;
    socklen_t client_len;
    struct sockaddr_in server, client;
    char *msg = "Hello\n";

    sd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    memset(&server, 0, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(SERVER_TCP_PORT);
    server.sin_addr.s_addr = htonl(INADDR_ANY);
    bind(sd, (struct sockaddr *)&server, sizeof(server));
    listen(sd, 5);
```

```

    printf("Server is waiting on port 3000...\n");

    while(1) {
        client_len = sizeof(client);
        new_sd = accept(sd, (struct sockaddr *)&client, &client_len);
        printf("Client connected! Sending Hello...\n");

        send(new_sd, msg, strlen(msg), 0);
        sleep(1);
        close(new_sd);
    }
}

```

Explanation: This segment of the code implements a tcp server that waits for a client to connect, then sends the message “Hello” to the client before closing the connection. This is done by defining a port number to be used by the server (in this case, listening for TCP connections on port 3000), declaring variables for the server socket descriptor, the client address length, the server address, and the message to be sent to the client. The socket is then created and configured to allow its reuse before being configured for the server address, and bound for the server’s address and port number. With the setup complete, the socket is put in passive mode with listen to allow it to accept incoming connections before entering a while loop where it continues to accept incoming connection requests and makes new sockets to store and interact with multiple clients before sending its “Hello” message upon successful connection. Lastly, the server pauses, then closes the connection.

hello_client.c

```

#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main(int argc, char **argv) {
    int sd, n;
    struct sockaddr_in server;
    char buf[100];

    sd = socket(AF_INET, SOCK_STREAM, 0);
    memset(&server, 0, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(3000);
    inet_aton(argv[1], &server.sin_addr);
}

```

```

    connect(sd, (struct sockaddr *)&server, sizeof(server));

    n = recv(sd, buf, 99, 0);
    if (n > 0) {
        buf[n] = '\0';
        printf("Message from server: %s", buf);
    }

    printf("Connection closed by server. Exiting.\n");
    close(sd);
    return 0;
}

```

Explanation: This segment of the code implements a TCP client that connects to the server, receives the “Hello” message from the first segment, displays it using the terminal and exits after the server closes the connection. This is done by declaring variables for the socket, a variable “n” to be used for the number of bytes received, server to store the address information, and a buffer to store the message received from the server. Creating a socket to bind to the server, and configuring its port number and converting its network byte order, before configuring the server address. Next connect() is used to establish a TCP connection with the server before the client uses recv() to receive the message before displaying it in the terminal and terminating the connection.

2. Part 3: TCP File Download Application

```

file_server.c

C
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define SERVER_TCP_PORT 3000

int main() {
    int sd, new_sd, fd, n;
    socklen_t client_len;
    struct sockaddr_in server, client;
    char filename[100], buf[101];

    sd = socket(AF_INET, SOCK_STREAM, 0);

```

```

int opt = 1;
setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

memset(&server, 0, sizeof(server));
server.sin_family = AF_INET;
server.sin_port = htons(SERVER_TCP_PORT);
server.sin_addr.s_addr = htonl(INADDR_ANY);
bind(sd, (struct sockaddr *)&server, sizeof(server));
listen(sd, 5);

printf("File Server is waiting on port 3000...\n");

while(1) {
    client_len = sizeof(client);
    new_sd = accept(sd, (struct sockaddr *)&client, &client_len);

    n = read(new_sd, filename, 99);
    filename[n] = '\0';
    printf("Client requested file: %s\n", filename);

    fd = open(filename, O_RDONLY);
    if (fd < 0) {
        buf[0] = '1';
        strcpy(buf + 1, "File not found.");
        write(new_sd, buf, strlen(buf));
    } else {
        while ((n = read(fd, buf + 1, 100)) > 0) {
            buf[0] = '0';
            write(new_sd, buf, n + 1);
        }
        close(fd);
    }
    close(new_sd);
}
}

```

Explanation: The server listens on port 3000 and accepts incoming TCP connections. When a client connects, it reads the requested filename, then attempts to open that file. If the file exists, it reads the requested filename, then attempts to open that file. If the file exists, it reads and sends the contents in 100-byte chunks, prefixing each packet with '0' to signal valid data; if the file cannot be opened; it sends an error message prefixed with '1'. The connection is closed after the file or error is transmitted.

file_client.c

```

C
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>

```



```

#include <arpa/inet.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main(int argc, char **argv) {
    int sd, n;
    struct sockaddr_in server;
    char filename[100], buf[101];

    sd = socket(AF_INET, SOCK_STREAM, 0);
    memset(&server, 0, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(3000);
    inet_aton(argv[1], &server.sin_addr);
    connect(sd, (struct sockaddr *)&server, sizeof(server));

    printf("Enter filename to download: ");
    scanf("%s", filename);
    write(sd, filename, strlen(filename));

    while ((n = read(sd, buf, 101)) > 0) {
        if (buf[0] == '1') {
            buf[n] = '\0';
            printf("\nSERVER ERROR: %s\n", buf + 1);
        } else {
            write(1, buf + 1, n - 1);
        }
    }
    printf("\nTransfer complete. Closing connection.\n");
    close(sd);
    return 0;
}

```

Explanation: The client connects to the server at the IP address provided as a command-line argument on port 3000. It prompts the user to enter a filename, sends it to the server, then reads the incoming packets in a loop. Each packet's first byte is checked: '1' indicates an error message (printed to the terminal), while '0' indicates file data (written to standard output). The loop exits when the server closes the connection.

3. Part 4: UDP File Download Application

udp_file_server.c

```

C
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <string.h>

```

```

#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_UDP_PORT 3001

struct pdu {
    char type;
    char data[100];
};

int main() {
    int sd, fd, n;
    struct sockaddr_in server, client;
    socklen_t client_len;
    struct pdu spdu, rpdu;
    struct stat file_stat;

    sd = socket(AF_INET, SOCK_DGRAM, 0);
    memset(&server, 0, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = htonl(INADDR_ANY);
    server.sin_port = htons(SERVER_UDP_PORT);
    bind(sd, (struct sockaddr *)&server, sizeof(server));

    printf("UDP File Server listening on port 3001...\n");

    while(1) {
        client_len = sizeof(client);
        recvfrom(sd, &rpdu, sizeof(rpdu), 0, (struct sockaddr *)&client,
        &client_len);

        if (rpdu.type == 'C') {
            printf("Client requested file: %s\n", rpdu.data);

            fd = open(rpdu.data, O_RDONLY);
            if (fd < 0) {
                spdu.type = 'E';
                strcpy(spdu.data, "Error: File not found or cannot be
opened.");
                sendto(sd, &spdu, sizeof(spdu.type) + strlen(spdu.data), 0,
                (struct sockaddr *)&client, client_len);
            } else {
                stat(rpdu.data, &file_stat);
                int total_size = file_stat.st_size;
                int bytes_read = 0;

                while ((n = read(fd, spdu.data, 100)) > 0) {
                    bytes_read += n;
                    if (bytes_read >= total_size) {
                        spdu.type = 'F';
                    } else {
                        spdu.type = 'D';
                    }
                }
            }
        }
    }
}

```

```

        }
        sendto(sd, &spdu, n + 1, 0, (struct sockaddr *)&client,
client_len);
        }
        close(fd);
    }
}
}
}

```

Explanation: The server binds to UDP port 3001 and waits for incoming PDU's from clients. When a Filename PDU (**type = 'C'**) is received, it attempts to open the requested file, if the file is missing, it sends an Error PDU back to the client. If the file, it reads and sends the contents in 100-byte Data PDUs (**'D'**), marking the last chunk as a Final PDU (**'F'**) using the file's total size obtained via **stat**.

udp_file_client.c

```

C
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

struct pdu {
    char type;
    char data[100];
};

int main(int argc, char **argv) {
    int sd, choice, n;
    struct sockaddr_in server;
    struct pdu spdu, rpdu;

    sd = socket(AF_INET, SOCK_DGRAM, 0);
    memset(&server, 0, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_port = htons(3001);
    inet_aton(argv[1], &server.sin_addr);

    while (1) {
        printf("\n1. Download a file\n2. Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 2) break;

        printf("Enter filename: ");
    }
}

```

```

        scanf("%s", spdu.data);
        spdu.type = 'C';

        sendto(sd, &spdu, strlen(spdu.data) + 2, 0, (struct sockaddr
*)&server, sizeof(server));

        FILE *fp = fopen("udp_download.txt", "wb");

        while (1) {
            n = recvfrom(sd, &rpdu, sizeof(rpdu), 0, NULL, NULL);
            if (rpdu.type == 'E') {
                rpdu.data[n-1] = '\0';
                printf("\nSERVER ERROR: %s\n", rpdu.data);
                fclose(fp);
                remove("udp_download.txt");
                break;
            } else if (rpdu.type == 'D') {
                fwrite(rpdu.data, 1, n - 1, fp);
            } else if (rpdu.type == 'F') {
                fwrite(rpdu.data, 1, n - 1, fp);
                printf("\nTransfer complete! Saved as
'udp_download.txt'\n");
                fclose(fp);
                break;
            }
        }
    }
    close(sd);
    return 0;
}

```

Explanation: The client presents a menu allowing the user to download a file or quit. When a filename is entered, it is sent to the server as a Filename PDU (type = “c”). The client then loops on **recvfrom**, writing received Data and Final PDU payloads to a local file (**udp_download.txt**), stopping on a Final PDU or displaying the error message and deleting the partial file on an Error PDU.